

Azure Horizon Resort — 15-Minute Group

Presentation Script

10 presenters • 12 use cases • Web + Mobile (one Firebase backend) • Each speaker 45–90 seconds • Spoken-word script — rehearse aloud

Run Sheet (who, what, when)

Clock	Presenter	Use Case
0:00–0:30	Bandile Maqeda	Opening — system overview & cross-platform framing
0:30–2:00	Fanelesibonge Mdlalose	UC23 — Book Event Venue
2:00–3:30	Lunga Bradley Gwala	UC24 — Book Event Catering
3:30–4:45	Bandile Maqeda	UC22 — Redeem Loyalty Points
4:45–6:15	Mpho Dlamini	UC25 — Send Event Invitations
6:15–7:45	Sinakhokonke Mwandla	UC26 — Conduct Pre-Event Inspection
7:45–9:15	Sifundo Hlambisa	UC27 — Check In Attendees
9:15–10:00	Lunga Bradley Gwala	UC28 — File Live Event Complaint
10:00–11:15	Melusi Mabanga	UC29 — Inspect Venue & Record Damages
11:15–12:00	Isaac Toluwanimi Olatunji	UC30 — Resolve Damage Claims
12:00–12:45	Simphiwe Owethu Bande	UC32 — Request Event Refund
12:45–13:15	Asanda S. Nzama	UC33 — Review Refund Request
13:15–14:00	Bandile Maqeda	UC31 — Submit Event Feedback
14:00–14:30	Flex — absorb running-over time here, then reset at 14:30	
14:30–15:00	Asanda S. Nzama	Closing — the engineering story

Cross-platform callouts: marked with a green box — these are the strongest architectural talking points. Only **five** screen actions across the whole presentation (yellow boxes).

Opening — Bandile Maqeda (0:00–0:30)

"Good day, lecturers. We're Team Azure Horizon. What we're about to show you is not ten separate apps — it's **one system with two faces**: a web portal and a mobile app that talk to a single Firebase backend. Every event you'll see today — the booking, the catering, the invitations, the inspection, the money — is the **same data, live on both platforms, the moment it's written**. The guest journey we'll walk you through starts with a venue booking — so I'll hand over to Fanelesibonge."

UC23 — Book Event Venue — Fanelesibonge Mdlalose (0:30–2:00)

[ACTION 1] On the web, guest account: Book an Event → choose the Grand Ballroom → pick a date → Confirm Booking → Pay Deposit. Then pick up the phone and open the same guest's bookings in the mobile app — the new reservation is already there.

"Guests book a venue through the web app — the Ballroom, the estate, the vineyards — each with capacity, pricing, and photos. The moment you pick a date, the system checks availability **against both date models the apps use**, so a date booked on a phone is instantly blocked on the web, and vice versa — no double-bookings, with a buffer slot for setup and teardown. When the booking is confirmed it's written as **pending payment**, with a guard that stops the same guest reserving the same date twice. Then the deposit screen takes over — and here's the part I love: the payment isn't a simulation. It **persists to the booking record** — the balance is re-derived, the folio is written, and the booking is confirmed.

CROSS-PLATFORM: this booking, its deposit, its balance — all visible on the phone right now, no refresh, no sync job.

Handoff → Lunga: "...and the moment that deposit clears, the system re-totals the event and offers the host the next natural step: catering."

UC24 — Book Event Catering — Lunga Bradley Gwala (2:00–3:30)

"Exactly — and catering isn't a separate invoice. When the host adds plated menus or buffets, the app **folds the catering cost into one combined event total** — venue plus food — with a single 50% deposit. The totals are re-calculated and re-saved on the booking, so every screen from here on derives the balance from **one source of truth**, never from stale local state. The caterer also gets an **ownership check** — you can only add catering to your own booking. And the payment prompt is honest: pay the deposit, pay in full, or settle it later — the balance just follows you to the dashboard."

Handoff → Bandile: "...and every rand paid earns loyalty points — which is where Bandile picks up."

UC22 — Redeem Loyalty Points — Bandile Maqeda (3:30–4:45)

[ACTION 2] Show the mobile loyalty card (points/tier), then the web Loyalty Store: redeem a voucher — points move to "held", a voucher code is issued.

"Loyalty is woven through the whole system, not bolted on. Each event payment earns the guest **one point per R10** — the same rule on both apps — and a **loyalty log** records every credit so points can't be double-awarded. Here the guest redeems from the store: the points **move into a held state** — they're not spent, they're reserved against the voucher. That voucher carries a signed code staff can scan at the front desk, and if the redemption is ever reversed, the points return to the balance. One ledger, two apps.

CROSS-PLATFORM: the points earned from that web deposit appeared on the mobile card instantly — same Firestore document.

Handoff → Mpho: "...with the venue booked, catering on the table, and the host's loyalty growing — the next job is telling the guests."

UC25 — Send Event Invitations — Mpho Dlamini (4:45–6:15)

"The RSVP dashboard is capacity-aware — it blocks any invitation batch that would exceed the attendance the host booked. Each guest is invited **by name**, and every invitee receives a **signed QR pass by email** — signed, so it can't be forged, and validatable by any scanner, web or mobile. The system also refuses duplicates: invite the same email again and it **reuses the existing pass** and simply re-dispatches the reminder — no double passes, no double entries. The host can open the Guest Passes view and see every attendee's card with their name and unique pass ID."

Handoff → Sinakhokonke: "...so the passes are out there in guests' inboxes — but before a single attendee walks through the door, the venue itself has to earn its approval. That's the pre-event inspection."

UC26 — Conduct Pre-Event Inspection — Sinakhokonke Mwandla (6:15–7:45)

[ACTION 3] On the **mobile app** (staff): Event Ops → Pre-Event Inspection → asset checklist → Submit (Approved).

"On event day, the venue inspector runs a **standardised asset checklist** per venue — flooring, furniture, AV, grounds — and grades each item. The inspection is written to its own collection and **gates everything that follows**: attendee check-in is refused until the venue's status is **Venue Approved for Guests**. An inspection that comes back needs-attention is reported to the event manager on the web dashboard, and the guest gets a notification with the result — so the whole chain is auditable. The inspection happens on the phone, in the venue, with the checklist in hand —

CROSS-PLATFORM: — and the approval instantly appears on the web Event Manager dashboard. Same document, both worlds.

Handoff → Sifundo: "...and once that status flips to approved, the doors can open — which is Sifundo's moment."

UC27 — Check In Attendees — Sifundo Hlambisa (7:45–9:15)

[ACTION 4] Mobile: Attendee Check-In → scan the QR pass from the Guest Passes screen → success with attendee name + check-in time. (Rescan to show the "already checked in" guard.)

"At the door, staff scan the pass with the mobile app. The QR payload is **validated against Firestore** — signature, invitation, event. A valid scan writes a **checked-in timestamp**, so re-scanning the same pass immediately reports 'already checked in' — that's how we stop one pass being shared around the room. Declined or missing invitations are rejected at the gate, and there's a manual search by name or email as a fallback. The same validation service powers the **web staff QR tool** too — the exact same check-in logic runs in the browser.

CROSS-PLATFORM: every scan lands in one live attendance feed — the web dashboard counts attendees as the mobile scans them in, in real time.

Handoff → Lunga: "...and once the event is live, guests can escalate in the moment if something isn't right."

UC28 — File Live Event Complaint — Lunga Bradley Gwala (9:15–10:00)

"During the event itself, a guest can file a **live complaint from their phone** — category, urgency, photos. It lands in a live complaints queue that staff work in real time: each complaint carries an **urgency level and a priority score**, and staff advance it from received to acknowledged to resolved. Resolving a complaint **notifies the guest immediately** — and everything stays attached to the booking for the record. That live-ops loop closes with the event itself — and then the venue has to answer for how it was treated."

Handoff → Melusi: "...when the last guest leaves, the post-event inspection begins — Melusi takes it from here."

UC29 — Inspect Venue & Record Damages — Melusi Mabanga (10:00–11:15)

[ACTION 5] Web: Event Manager Dashboard → Start Inspection on the event → flag 1–2 assets as damaged/missing with photo + estimated cost → Submit.

"Post-event, the event manager runs the same **standardised venue checklist** in reverse — but now every asset is graded good, damaged, or missing. Anything flagged needs **a description, a photo, and an**

estimated repair cost — photo evidence, not hearsay. Submission writes a full damage record with the inspector, the items, and the total, and **updates the booking itself** with the damage penalty total — so the money trail stays on one document. Staff are even blocked from inspecting an event they organised themselves, and the same inspection flow exists on the mobile app for on-site use.

CROSS-PLATFORM: the report submitted here on the web is immediately visible in the mobile event ops screen — the guest is notified too.

Handoff → Isaac: "...that damage record doesn't sit in a drawer — it becomes a claim that someone has to adjudicate."

UC30 — Resolve Damage Claims — Isaac Toluwanimi Olatunji (11:15–12:00)

"Every post-event inspection that finds problems automatically feeds the **damage claim queue**. The admin reviews the inspector's record — the flagged assets, photos, and estimated costs — and decides: enforce the full claim, or **grant a partial or full waiver** where there's pre-existing wear, or where the guest is a valued loyalty member. The decision is recorded against the claim, and the outcome flows straight into the guest's folio — the damage amount is **linked to the same booking record**, so when money moves later, nothing is double-counted."

Handoff → Simphiwe: "...and sometimes the guest has their own grievance — they believe they're owed money back."

UC32 — Request Event Refund — Simphiwe Owethu Bande (12:00–12:45)

"From the mobile app, a guest can submit a **refund request against an event booking** — the amount they're claiming and their reason. The request is saved against their booking with a clear status, so they always know where it stands. But a refund request doesn't just appear on someone's desk — it enters a **review pipeline that knows the whole story**: if that same booking carries an unresolved damage claim, the reviewer is warned about the conflict before any money is returned. And the review itself is Asanda's job."

Handoff → Asanda: "...and every one of those requests lands on Asanda's review screen."

UC33 — Review Refund Request — Asanda S. Nzama (12:45–13:15)

"The admin's refund queue doesn't just list requests — it **scores them**. Each request is rated against five risk heuristics — how the claim size compares to the resort average, prior refund history, claimed percentages, and more — producing a **0-to-100 risk score** to guide the decision. Approving dispatches a refund invoice to the guest; rejecting requires a reason that's recorded. So the decision isn't a gut call — it's transparent, documented, and cross-checked against damage claims."

Handoff → Bandile: "...and when the money's settled and the event is truly over, the guest gets one last chance to be heard."

UC31 — Submit Event Feedback — Bandile Maqeda (13:15–14:00)

"The guest journey closes with **event feedback** — rating the venue, the catering, the overall experience, and leaving a written review. Feedback lands in the **community reviews** the resort publishes and in the admin's review manager, where staff can respond. What I want to emphasise: feedback isn't isolated — it's tied to the same booking, so when the resort reads 'the sound system let us down', they can look back at the actual event record, the complaints, and the inspection results, and fix the real problem."

Handoff → Asanda: "...and that's the full lifecycle — from the first venue click to the closing review. Asanda, over to you."

Closing — Asanda S. Nzama (14:30–15:00)

"To close: what you've seen isn't twelve features — it's **one connected system**. Every use case writes to the same Firestore documents, and both platforms derive their state from those documents in real time — the booking, the money, the inspections, the claims, the refunds, all one record. The technical ideas we'd ask you to remember: a **single source of truth** for financial state, **signed, validatable QR passes**, and a **cross-platform architecture** where the web and mobile apps are interchangeable front-ends to one backend. Thank you — we're happy to take questions."

Presenter Notes

Before you start — have open and ready:

- **Web (browser, signed in as guest):** Guest portal home — one click from "Book an Event"; a second tab signed in as **Event Manager** (Event Manager Dashboard) and a third as **Admin** (Refund Requests / Damage Claims tabs).
- **Mobile (phone):** signed in as the same guest, on the reservations screen; staff profile signed in for Event Ops / Attendee Check-In. Have one **sample event booking** with a QR pass already generated, and a couple of invitee emails that actually receive the EmailJS dispatch.
- **Screen mirroring:** phone mirrored to the projector so [ACTION 1–4] are visible without fumbling. Rehearse the mirror switch at least twice — it's the biggest time sink.

If you're running over: the flex block is at 14:00–14:30. Cut first: Lunga's UC28 (45s), Bandile's UC31 (45s), then trim the opening to one line each for Bandile/Asanda. Never cut [ACTION 1], [ACTION 4], or [ACTION 5].

The three moments to defend in Q&A:

- **Live cross-platform sync (Fanelesibonge, [ACTION 1]):** the web booking is visible on mobile instantly because both apps read the same Firestore document — there is no separate mobile database and no sync job.
- **Financial integrity (Lunga, Bandile):** payments accumulate on the booking record and every balance is re-derived from it — the client never "tells" the server what the balance is; combined event totals keep venue + catering + payments on one ledger.
- **Signed QR passes & the gated chain (Mpho → Sinakhokonke → Sifundo):** invites are signed invitation records, check-in is refused until pre-inspection approval, and re-scans are flagged — a genuine anti-fraud chain, not three bolted-on screens.

Platform honesty: UC28 (live complaint) and UC32 (refund request) are demonstrated from the **mobile** app; UC33 review and UC29 post-inspection run on the **web**. If a lecturer asks why a use case lives on one platform and not the other, the answer is deliberate: each use case is shown on the platform where it's strongest, and the shared backend means every one of them is fully reflected on both. Say it with confidence — it's an architecture decision, not an omission.